

2η σειρά γραπτών ασκήσεων Αλγόριθμοι και Πραγματοποιήσεις

Άσκηση 1: Παιχνίδι με Κάρτες

Έχουμε ακολουθία n καρτών σε ευθεία, κάθε μία εκ των οποίων έχει ένα θετικό αμέριστο α. Το παιχνίδι παίζουν εναντίον δύο παίκτες, πρώτα ο X και μετά ο Y , επιλέγοντας κάθε φορά τη βέλτιστη κάρτα μεταξύ της $1ης$ και της τελευταίας που υπάρχει στην ευθεία. Η ευθεία, με την ικανότητα να "προβλέπουν" γίνεται {προστα.

Σε κάθε γύρο ο τρέχων παίκτης προσπαθεί να βελτιστοποιήσει το συνολικό κέρδος του και να ελαχιστοποιήσει το ~~α~~ συνολικό κέρδος του αντίπαλου του. Επομένως η αναδρομική σχέση θα έχει συνδυασμό $\max$ - $\min$, όπου το $\max$ θα είναι η τρέχουσα βέλτιστη επιλογή και το $\min$ θα είναι η προηγούμενη χειρότερη επιλογή που θα έχει αφήσει ο αντίπαλος παίκτης για τον τρέχον^{ον} παίκτη. Η αναδρομική σχέση επομένως θα έχει την εξής μορφή:

$$S(i, j) = \begin{cases} a_i & i > j \text{ και } i, j = 1, \dots, n \\ a_j & i = j \text{ και } i, j = 1, \dots, n \\ \max \{ a_i, a_j \} & i \neq j \text{ και } |j - i| = 1 \text{ και } i, j = 1, \dots, n \\ \max \{ a_i + \min \{ S(i+1, j-1), S(i+2, j) \}, a_j + \min \{ S(i, j-2), S(i+1, j-1) \} \} & i \neq j \text{ και } |j - i| > 1 \text{ και } i, j = 1, \dots, n \end{cases}$$

Ο αλγόριθμος διατυπώνεται σε μορφή ψευδοκώδικα ως εξής:

```
for i ← 1 to n do
  for j ← 1 to n do
    if i > j then
      S(i, j) ← 0;
    else if i = j then
      S(i, j) ← ai
```

```
return (s(l, n));
```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγορίθμου, έχουμε δύο επεκτατικότερα loops από το 1 έως το n . Επομένως ο αλγόριθμος θα έχει υπολογιστική πολυπλοκότητα $O(n^2)$.

Αναφορικά με την αρθρότητα του αλγορίθμου, θα την αποδείξω χρησιμοποιώντας επαγωγή. Αν έχουμε μια μόνο κάρτα ($i=j$), προφανώς θα επιλέξει αυτή. Αν έχουμε δύο κάρτες, θα επιλέξει η βέλτιστη από αυτές. Προφανώς για $i > j$ δεν ορίζεται επιλογή. Έστω για την γενική επιλογή, έχουμε τα ακόλουθα 2 ενδεχόμενα: Είτε επιλέγουμε την 1η κάρτα (a_i), είτε την τελευταία (a_j). Για την 1η περίπτωση, ο αντίπαλος θα πρέπει να επιλέξει ανάμεσα ^{ανάμεσα} ~~ανάμεσα~~ σε a_{i+1} και a_j κάρτων. Προφανώς θα επιλέξει τη βέλτιστη για αυτόν ($\max$), αφήνοντας σε εμάς στον επόμενο γύρο τη χειρότερη δυνατή επιλογή. Συμμεριφόμενα, είτε θα επιλέξει την a_{i+1} , αφήνοντας μας το $\min S(i+1, j)$, είτε την a_j , αφήνοντας μας το $S(i+1, j-1)$, το ελάχιστο εκ των δύο. Για την 2η περίπτωση, ο αντίπαλος επιλέγει ανάμεσα σε a_i και a_{j-1} . Αυτή τη φορά θα επιλέξει είτε την a_i , αφήνοντας μας το $S(i+1, j-1)$, είτε την a_{j-1} , αφήνοντας μας το $S(i, j-1)$, πάλι το ελάχιστο εκ των δύο. Η διαφορά μας επιλογή είναι το μέγιστο των 2 δυνάμεις μας επιλογών, λαμβάνοντας υπόψη τις επιλογές ~~και~~ ^{αυτού} του αντίπαλου μας αφήνει ο αντίπαλος. Επομένως ο αλγόριθμος είναι ορθός.

Άσκηση 2: Ταξιδεύοντας στον κόσμο

$G(V, E, \bar{c}, \tau)$ μη κατευθυνόμενο γραφικό με n κορυφές, m ακμές και θετική αλγεαία κόστος $\bar{c}$ και τ στις ακμές. $c(e)$ το κόστος ετήσιας κάρτας, $\tau(e)$ κόστος αλλαγής μεταφοράς πάνω στην ακμή e . Αναζητάμε το συντομότερο s - t μονοπάτι, με βάση το κόστος c , που ελαχιστοποιεί το κόστος τ μεταξύ των πόλεων a - b .

Εφόσον έχουμε γραφικό με θετική τιμή, ο γρηγορότερος αλγόριθμος που μπορούμε να χρησιμοποιήσουμε για την εύρεση των συντομότερων s - t μονοπατιών είναι ο Dijkstra. Επιπλέον, εφόσον το γραφικό είναι πυκνό, θα υλοποιήσουμε τον Dijkstra γραμμικά.

Αρχικά, θα τρέξουμε Dijkstra στο γραφικό G , ώστε να βρούμε όλα τα συντομότερα s - t μονοπάτια. Υστερα, θα τρέξουμε τον αλγόριθμο Dijkstra από την κορυφή a προς όλες τις άλλες κορυφές και από την κορυφή b προς όλες τις άλλες κορυφές, με βάση το κόστος τ . Το μονοπάτι s - t που θα επιλέξουμε, θα είναι το συντομότερο s - t μονοπάτι που θα ελαχιστοποιεί το ~~αθροισμα~~ αθροισμα από το a σε αυτό με βάση το κόστος τ , και από το b σε αυτό, ομοίως. Οι ακμές που ανήκουν στο μονοπάτι s - t θα έχουν προφανώς ~~από~~ $c(e) = 0$. Ο αλγόριθμος διατυπώνεται σε ψευδοκώδικα ως εξής:

for all $u \in V$ do

$D[u] \leftarrow \infty$;

$p[u] \leftarrow \text{NULL}$;

$D[s] \leftarrow 0$;

$S \leftarrow \emptyset$;

while $|S| < |V|$ do

$u \in S : D[u] = \min_{v \notin S} \{D[v]\}$;

```

 $S \leftarrow S \cup \{u\}$ ;
for all  $v \in \text{AdjList}[u]$  do
    if  $D[v] > D[u] + c(u,v)$  then
         $D[v] \leftarrow D[u] + c(u,v)$ ;
         $p[v] \leftarrow u$ ;
for all  $u \in V$  do
     $A[u] \leftarrow \infty$ ;
     $p[u] \leftarrow \text{NULL}$ ;
 $A[a] \leftarrow 0$ ;
 $A \leftarrow \emptyset$ ;
while  $|A| < |V|$  do
     $u \leftarrow \arg \min_{v \in S} A[v]$ ;
     $A \leftarrow A \cup \{u\}$ ;
    for all  $v \in \text{AdjList}[u]$  do
        if  $A[v] > A[u] + \tau(u,v)$  then
             $A[v] \leftarrow A[u] + \tau(u,v)$ ;
             $p[v] \leftarrow u$ ;
        }
    to ιδίω για  $B[b]$  και  $p[b]$  αντιστοίχα
for all  $e \in P$  (P το σύνολο των συντομότερων s-t μονοπατιών)
total-cost  $\leftarrow \underbrace{c(e)}_{\text{all s-t edges}} + \underbrace{\tau(u,v)}_{\text{min from a}} + \underbrace{\tau(v)}_{\text{min from b}}$ 
for all  $v \in V$  do
total-cost  $\leftarrow D[v] + A[v] + B[v]$ 
return (shortest s-t path with min(total-cost))

```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, έχουμε $O(n^2)$ για κάθε Dijkstra, εφόσον το γεγονός είναι πυκνό και $O(n)$ για τον υπολογισμό του ελάχιστου υψους. Επομένως ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(m^2)$.

Αναφορικά με την πρέμια του αλγόριθμου, γνωρίζουμε ότι ο Dijkstra βρίσκει τα συντομότερα μονοπάτια με βάση τα υψών ~~και~~ αντιστοίχα. Εφόσον παίρνουμε το συντομότερο s-t μονοπάτι που να ελαχιστοποιεί το υψος

Το μεταξύ των πόλων $a-b$, οι 3 ανεξάρτητοι
Dijkstra εφαρμόζονται τα συντομότερα μονοπάτια
από τις 3 αλυσίδες (s, a, b). Έπειτα, επιλέγοντας
από τα συντομότερα μονοπάτια αυτό με το ελάχιστο
συνολικό κόστος, όπου στο συνολικό κόστος υπολογίζεται
το $c(e)$ για τις ακμές του $s-t$ μονοπατιού και
το $\tau(e)$ για τις ακμές από το a μέχρι την κορυφή
του $s-t$ μονοπατιού με το μικρότερο $A[v]$ κόστος
μαζί και από το b μέχρι την κορυφή του $s-t$
μονοπατιού με το μικρότερο $B[v]$ κόστος, εφαρμόζου-
με ότι το επιλεγμένο $s-t$ μονοπάτι πληρεί όλες
τις απαραίτητες προϋποθέσεις. Επιπλέον, ο αλγόριθμος
είναι ορθός.

~~Σε κάθε επέκταση επιλογή, τα $D[v]$ και $T[v]$ είτε θα μειώνονται, είτε θα παραμένουν σταθερά, επομένως όταν το S θα δίνει ίδιο με το V , το $D[b]$ θα είναι το συντομότερο $s-t$ μονοπάτι, το οποίο θα ελαχιστοποιεί το $T[b]$, δηλαδή το κόστος με βάση το τ για το μονοπάτι $a-b$.
Συνεπώς, ο αλγόριθμος είναι ορθός.~~

Άσκηση 3: Επίλυση συστήματος ανισοτήτων

$x_1, \dots, x_n$ αμέραις μεταβλητές. Έχουμε σύστημα S αποτελούμενο από m ανισότητες $x_i - x_j \leq b_{ij}$, $1 \leq i, j \leq n$, b_{ij} αμέραις αριθμοί. Σ υανοποιησιμο αν υπάρχουν αμέραις τιμές για τις μεταβλητές $x_1, \dots, x_n$ που ικανοποιούν όλες τις ανισότητες του S .

α) Μετατρέποντας τις ανισότητες στη μορφή $x_i \leq x_j + b_{ij}$, παρατηρούμε πως συμβολίζουν τη σύνδεση δύο κορυφών x_i, x_j ($x_j \rightarrow x_i$) με βάρος b_{ij} . Επομένως, μπορείτε να μετασχηματίσετε το πρόβλημα ικανοποισιμότητας του συστήματος S στο πρόβλημα αναζήτησης κύκλου αρνητικού μήκους σε γραφικό με κορυφές $x_1, \dots, x_n$ και αυτές τις ανισότητες $x_i - x_j \leq b_{ij}$, δηλαδή αυτές από τα x_j στα x_i με μήκος b_{ij} . Αν υπάρχει κύκλος αρνητικού μήκους, αυτό σημαίνει ότι υπάρχουν ανισότητες που οδηγούν σε αντίφαση, επομένως το S δεν είναι ικανοποιησιμο. Αν δεν υπάρχει κύκλος αρνητικού μήκους, οι τιμές των $x_1, \dots, x_n$ θα ισχύουν με τα $D[x_1], \dots, D[x_n]$ που αποσπεύει ο αλγόριθμος Bellman-Ford, όταν τρέφει, στο γραφικό αυτό, ξεκινώντας από κάποιο x_i κορυφή, έναν κόμβο s που συνδέεται με όλα τα $x_1, \dots, x_n$ με κόστος 0 .

Ο αλγόριθμος που επιλύει το πρόβλημα αυτό διατυπώνεται σε μορφή ψευδοκώδικα ως εξής:

```


V ← ∅;
E ← ∅;
for all  $x_i \rightarrow x_j = b_{ij} \in S$  do
    V ← V ∪ { $x_i$ };
E ←


```

```

for all  $u \in V$  do ( $V = \{x_1, \dots, x_n\}$ )
     $D[u] \leftarrow \infty$ ;
     $p[u] \leftarrow \text{NULL}$ ;
 $D[x_1] \leftarrow 0$ ;
for  $i \leftarrow 1$  to  $n$  do
    for all  $(u, v) \in E$  do ( $E = \{x_i, x_j : x_i - x_j \in b_{ij}\}$ )
        if  $D[u] > D[v] + b_{uv}$  then
             $D[u] \leftarrow D[v] + b_{uv}$ ;
             $p[u] \leftarrow v$ ;
for all  $(v, u) \in E$  do
    if  $D[u] > D[v] + b_{vu}$  then
        return ("η ικανοποίηση");
for  $j \leftarrow 1$  to  $n$  do
    return ( $D[x_j]$ );

```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγορίθμου, έχουμε $O(m)$ κόστος για τη δημιουργία του γράφου και $O(m \cdot n)$ για τον Bellman-Ford. Επομένως η υπολογιστική πολυπλοκότητα του αλγορίθμου είναι $O(m \cdot n)$.

Αναφορικά με την ορθότητα, αν ο γράφος έχει μηδενικούς αρνητικούς βάρους, αυτό σημαίνει ότι υπάρχει μια σειρά απόδοσης, οι οποίες οδηγούν σε αύξηση, ^{αλλά το αποτέλεσμα} άρα δε μπορεί να βρούμε τιμές για τις μεταβλητές που ικανοποιούν όλες τις ανισότητες ταυτόχρονα, ενδεώς το σύστημα S είναι η ικανοποίηση. Αν δεν υπάρχουν μισοί αρνητικοί βάρους το S είναι ικανοποίηση και οι τιμές $D[x_i]$, $i=1, \dots, n$ που αποθηκεύει ο Bellman.

-Ford ξεκινώντας από ένα τυχαίο κόμβο ικανοποιούν το σύστημα των ανισοτήτων ($D[x_i] \leq D[x_j] + b_{ij}$). Επομένως είναι αποδεδειγμένες τιμές για τις μεταβλητές $x_1, \dots, x_n$. Οπότε, ο αλγόριθμος είναι ορθός.

β) Η τροποποίηση του αλγορίθμου του α) ερωτάται, ώστε αν το σύστημα S δεν είναι ικανοποιήσιμο, να υπολογίζει ένα ελάχιστο ως προς το πλήθος των ανισοτήτων υποσύνστημα S' , η ικανοποιήσιμο, είναι ~~απόδοσης~~ η απόδοσή:

Καταρχήν, εισάγουμε ένα πίνακα E που ως τιμές έχει το πλήθος των ακμών που χρειάζεται για φτάσουμε στην ελάχιστη κορυφή και σε περίπτωση ισόβαθμιας $D[v]$, επιλέγεται και $D[u] + b_{uv}$, επιλέγεται το μονοπάτι με το ελάχιστο G . Αν ο Bellman-Ford στο τέλος του εντοπίσει κύκλο αρνητικού μήκους, τότε με βάση τον πίνακα p ανατρέχουμε όλες τις ακμές που συμμετείχαν στο μονοπάτι που βρέθηκε ο κύκλος αρνητικού μήκους και τις τοποθετούμε σε ένα νέο γράφο G' . Στο γράφο G' τρέχουμε BFS και για κάθε επίπεδο, ελέγχουμε πρώτα για ύπαρξη κύκλου και ύστερα αν υπάρχει κύκλος, αν είναι θετικού ή αρνητικού μήκους. Ο BFS μας εφασφαλίζει ότι ο 1ος κύκλος αρνητικού μήκους που θα βρούμε, θα είναι και αυτός με το λιγότερο πλήθος ακμών.

Ο τροποποιημένος αλγόριθμος αποτυπώνεται σε ψευδοκώδικα ως εξής:

for all $u \in V$ do ($V = \{x_1, \dots, x_n\}$)

$D[u] \leftarrow \infty;$

$p[u] \leftarrow \text{NULL};$

$E[u] \leftarrow 0;$

$D[s] \leftarrow 0;$

$E[s] \leftarrow 0;$

```

for i ← 1 to n-1 do
  for all (u,v) ∈ E do (E = {xi, xj} : xi - xj ≤ bij)
    if D[u] > D[v] + buv then
      D[u] ← D[v] + buv;
      p[u] ← v;
      E[u] ← E[v] + 1;
for all if D[u] = D[v] + buv and E[u] > E[v] + 1 then
      E[u] ← E[v] + 1; p[u] ← v;
  for all (u,v) ∈ E do and (u,v) not visited
    if D[u] > D[v] + buv then
      while p[u] ≠ NULL do
        add (u, p[u]) to G'; (u, p[u]) ← visited;
      run BFS to G'
      if you find cycle, check for the sum of
      bij for all (xi, xj) edges of the cycle.
      if 2 bij < 0 return all (xi, xj) of the
      cycle, else continue.
  for j ← 1 to n do
    return (D[xj]);

```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, έχουμε $O(n \cdot m)$ για τον Bellman-Ford, $O(m)$ για την υακατεύση του γράφου και $O(m) \cdot n$ για τον BFS. Επομένως, η υπολογιστική πολυπλοκότητα του αλγόριθμου είναι $O(n \cdot m)$.

γ) Για το πρόβλημα αυτό θα χρησιμοποιήσουμε αλγόριθμο παρόμοιο με αυτόν του β) ερωτήματος, με τη βασική διαφορά ότι πλέον συγγραφικά τα βάρη w_{ij} των ακμών και όχι το πηλίκο των ακμών των κινδύνων απεικονίζονται για την εύρεση ελάχιστου ποσυστημάτος S' που δεν είναι υαυο-ποινάτο. Συγκεκριμένα, θα ετεδεύουμε Bellman-Ford για το δένδρα των ανισοτήτων και θα ανσυνεύουμε και το ~~πρόσ~~ βάρος w_{ij} εκτός από το b_{ij} , και σε περίπτωση ισότητας b_{ij} , θα ετεδεύεται η ακμή

με το μικρότερο w_{ij} . Στην n -οστή επανάληψη του Bellman-Ford θα γίνεται έλεγχος για αρνητικούς κύκλους. Αν δε βρεθούν, θα επιστρέφονται οι τιμές $D[x_j]$, $j=1, \dots, n$ όπως στο α) ερώτημα, εφόσον το S θα είναι ικανοποιησιμο. Αν βρεθούν, με ανάστροφο στο $p[u]$ θα ανακατασκευάζεται ο κύκλος, θα απενευστεί το υπόλοιπο του με βάση το w_{ij} και αυτό θα στη συνέχεια θα αφαιρείται από το πρόβλημα. Αυτή η διαδικασία θα επαναλαμβάνεται έως ότου ελεγχθούν όλες οι απτες κορυφές για να εξασφαλιστεί η εύρεση αρνητικού κύκλου ελάχιστου συνολικού βάρους. Ο αλγόριθμος αυτός διατυπώνεται σε ψευδοκώδικα ως εξής:

```

for all  $u \in V$  do  $(CV = \{x_1, \dots, x_n\})$ 
     $D[u] \leftarrow \infty$ ;
     $p[u] \leftarrow \text{NULL}$ ;
     $C[u] \leftarrow \infty$ ;
 $D[S] \leftarrow 0$ ;  $\text{min\_cost} \leftarrow \infty$ ;
 $C[S] \leftarrow 0$ ;  $\text{best\_cycle} \leftarrow \emptyset$ 
for  $i \leftarrow 1$  to  $n-1$  do
    for all  $(u, v) \in E$  do  $(E = \{x_i, x_j : x_i - x_j \leq b_{ij}\})$ 
        if  $D[u] > D[v] + b_{uv}$  then
             $D[u] \leftarrow D[v] + b_{uv}$ ;
             $p[u] \leftarrow v$ ;
             $C[u] \leftarrow C[v] + w_{uv}$ ;
        if  $D[u] = D[v] + b_{uv}$  and  $C[u] > C[v] + w_{uv}$  then
             $C[u] \leftarrow C[v] + w_{uv}$ ;
             $p[u] \leftarrow v$ ;
for all  $(u, v) \in E$  do
    if  $D[u] > D[v] + b_{uv}$  then
        cycle  $\leftarrow$  extract cycle (αποδόχνησε τα  $p[u]$  μέχρι να γραφεί ο ίδιος κύκλος)
        cycle_cost  $\leftarrow \sum w_{ij}$  for all edges of cycle;
        if cycle_cost < min_cost then
            min_cost  $\leftarrow$  cycle_cost;
            best_cycle  $\leftarrow$  cycle;
    
```

```

return (best-cycle);
if (best-cycle not  $\emptyset$  then
    return (best-cycle);
else
    for  $j \leftarrow 1$  to  $n$  do
        return ( $D[x_j]$ );

```

Αναφορικά με την υποδοχιστική πολυπλοκότητα του αλγορίθμου, έχουμε $O(n \cdot m)$ για τον Bellman-Ford, $O(n)$ για την ανίχνευση και αφαίρεση του υαθέ υαυδου και $O(n)$ για την ευτύπωση των αποδέκων, τικών για τις μεταβάντες $x_i, i=1, \dots, n$. Επομένως, η υποδοχιστική πολυπλοκότητα του αλγορίθμου είναι $O(n \cdot m)$. Η πολυπλοκότητα αυτή είναι πολυωνυμική στο μέγεθος της εισόδου.

Άσκηση 4: Μετατροπή Ροής Ααυών σε Ροή Μονοπατιών

Έαυμε $s-t$ δαυτο $G(V, E, \bar{c})$ με n υαυχρές, m ααυές και θέσιμή ααέραια χωρητιότητα $c(e)$ σε υαθέ ααυή $e \in E$. Επαιτέον, έαυμε (ααέραια) $s-t$ ροή ααυών $f: E \rightarrow \mathbb{N}$ παυ αναθέτει ααέραια ροή $f(e), 0 \leq f(e) \leq c(e) \forall e \in E$ του G και έαυσαλίδει διατήρηση ροής σε υαθέ ενδίακση υαυρή, $v \in V \setminus \{s, t\}$.

Θεωραίτε P το σύνολο όλων των $s-t$ μονοπατιών στο G . Μια ααέραια ροή μονοπατιών $g: P \rightarrow \mathbb{N}$ αναθέτει ααέραια ροή $g(p)$ σε υαθέ $s-t$ μονοπάτι $p \in P: \sum_{p \in P} g(p) \leq c(e) \forall e \in E$. Η διατήρηση ροής στις ενδίακτες υαυρές έαυσαλίδεται γιατι η g έαυσαλίδεται ως ροή $s-t$ μονοπατιών.

Μια $s-t$ ροή ααυών $f: E \rightarrow \mathbb{N}$ αντιστοιχεί σε μια $s-t$ ροή μονοπατιών $g: P \rightarrow \mathbb{N}$ αν $f(e) = \sum_{p \in P: e \in p} g(p) \forall e \in E$.

Για το πρόβλημα της μεταρρύθμισης ροής αυτών σε ροή δυναμικών, θα χρησιμοποιήσουμε αλγόριθμο που υλοποιεί τη ροή $f(e)$ μαζί αυτών στη ροή του δυναμικού (η δυναμικών) $g(p)$, στο οποίο (στα οποία) ανήκει. Συγκεκριμένα, έχει να ελεγχθεί η ροή $f(e)$ αυτών, τρέχουμε DFS για την εύρεση $s-t$ δυναμικού, βρίσκουμε την ελάχιστη ροή ∞ από όλες τις αυρές του $f(e)$, την προσθέτουμε στο $g(p)$ του δυναμικού και την αφαιρούμε από όλες τις αυρές του. Η ροή $f(e)$ ελεγχθεί όταν δεν υπάρχει πλέον διαδρομή $s-t$ δυναμικού με $f(e) > 0$ σε όλες τις αυρές του. Ο αλγόριθμος αυτός διατυπώνεται σε ψευδοκώδικα ως εξής:

```

for all  $p \in P$  do
     $g(p) \leftarrow 0$ ;
while there is a path from  $s$  to  $t$  with  $f(e) > 0$  for all  $e \in p$  do
    run DFS and to find an  $s-t$  path with  $f(e) > 0$  for all  $e \in p$ ;
     $min \leftarrow \min_{e \in p} f(e)$ ;
     $g(p) \leftarrow g(p) + min$ ;
    for all  $e \in p$  do
         $f(e) \leftarrow f(e) - min$ ;
for all  $p \in P$  with  $g(p) > 0$  do
    return ( $g(p)$ );

```

Ο αλγόριθμος αυτός ελεγχθεί ότι $g(p) > 0$ σε m το πολύ δυναμικά του P ως εξής: Σε κάθε επανάληψη, βρίσκουμε ένα $s-t$ δυναμικό και προσθέτουμε στο $g(p)$ του ροή τουλάχιστον 1, λόγω της αμέγαιας φύσης της ροής. Η ροή αυτή και του περιορισμού για $f(e) > 0$ $\forall e \in p$. Η ροή αυτή ισούται με $\min_{e \in p} f(e)$ το οποίο σημαίνει ότι σε κάθε επανάληψη ^{επ} μινιμίζεται τουλάχιστον για ~~μία~~ αυρή του γραφήματος. Εφόσον έχουμε m αυρές, μπορούν να μινιμίσουν το πολύ m αυρές, επομένως να απουσιάσουν ροή $g(p) > 0$ το πολύ m δυναμικά.

Αναφορικά με την υποδοκιματική πολυπλοκότητα του αλγορίθμου, έχουμε $O(m)$ για την αρχικοποίηση των $g(p)$ (δεδομένου του περιορισμού των m βουνοπασιών), m το πολύ επαναλήψεις του DFS που υλοποιεί $O(n+m)$ και της ενημέρωσης των αμυνών που υλοποιεί $O(m)$, καθώς και $O(m)$ για την εκτύπωση των $g(p)$ (δεδομένου πάλι του περιορισμού). Επομένως, ο αλγόριθμος έχει χρονική πολυπλοκότητα ~~$O(m^2 + nm)$~~ $O(m^2 + nm)$.

Άσκηση 5: Επιβεβαίωση και Αναπροσαρμογή Μέγιστης Ροής

Έχουμε $s-t$ δίκτυο $G(V, E, \bar{c})$ με n κορυφές, m ακμές και θετική αμέγιστη χωρητικότητα $c(e)$

1. Για το πρόβλημα επαλήθευσης της μεγιστότητας ροής ροής f μεταξύ των κορυφών $s-t$ στο G , μπορείτε να χρησιμοποιήσετε τον ακόλουθο αλγόριθμο. Δημιουργείτε το υποδείγματικό δίκτυο $G_f(V, E_f, r_f)$ για το γράφο $G(V, E, \bar{c})$ και χρησιμοποιώντας DFS ελέγχετε για την ύπαρξη επαυξητικού ροοπατίου από την κορυφή s στην κορυφή t . Αν υπάρχει τέτοιο ροοπάτι, η ροή f αυτή δεν είναι μέγιστη, ενώ αν δεν υπάρχει, η ροή f είναι μέγιστη. Ο αλγόριθμος αυτός διατυπώνεται σε ψευδοκώδικα ως εξής:

```
for all  $(u, v) \in E$  do
    * if  $f(e) > 0$  then
        if  $c(e) - f(e) > 0$  then
             $E_f \leftarrow E_f \cup \{(u, v)\}$ ;
            *  $r_{uv} \leftarrow c(e) - f(e)$ ;
             $E_f \leftarrow E_f \cup \{(v, u)\}$ ;
             $r_{vu} \leftarrow f(e)$ ;
run DFS on  $G_f$  from  $s$ ;
if a path from  $s$  to  $t$  is found then
    return (not Max flow);
else
    return (Max flow);
```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, χρειάζεστε $O(m)$ για την κατασκευή του υποδείγματικού δικτύου και $O(m)^{1+n}$ για τον DFS. Επομένως, ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(m)^{1+n}$ γραμμικού χρόνου, όπως φαίνεται από την εξίσωση.

Αναφορικά με την ορθότητα του αλγόριθμου, έχουμε τα εξής: Το υποδειγματικό δίκτυο G_f αναπαριστά μέσω των ακμών του τη δυνατότητα περεαίρω αυτών της ροής σε κάθε ακμή. Συγκεκριμένα, η μπροστά ακμή $(c(e) - f(e))$ δείχνει πόσο ακόμη μπορεί να αυξηθεί η ροή μέχρι να γεμίσει τη μέγιστη χωρητικότητα της, ενώ η πίσω ακμή την τρέχουσα χωρητικότητά της. Μέγιστη ροή σημαίνει $c(e) = f(e)$ σε τουλάχιστον μια ακμή σε κάθε $s-t$ μονοπάτι, ειδικάως θα μπορούσαμε να αυξήσουμε τη ροή με βάση τον αλγόριθμο Ford-Fulkerson. Αυτό συνεπάγεται τουλάχιστον μια ακμή σε κάθε $s-t$ μονοπάτι που έχει μόνο πίσω ακμή, διακόπτοντας τα $s-t$ μονοπάτια στο G_f . Επομένως, ύπαρξη επαυξητικού μονοπατιού στο G_f σημαίνει όχι μέγιστη ροή, μη ύπαρξη σημαίνει μέγιστη και ο αλγόριθμος είναι ορθός.

2. Θεωρώντας ότι η f είναι μέγιστη $s-t$ ροή στο G , ανααλύουμε ότι η χωρητικότητα $c'(e)$ μιας ακμής e μπορεί κατά $k \geq 1$ μονάδες σε σχέση με τη χωρητικότητά $c(e)$ που θεωρήσαμε αρχικά. Φανταστείτε αλγόριθμο που τροποποιεί τη μη αποδεδειγμένη f σε f' αποδεδειγμένη και μέγιστη ροή f' για το δίκτυο G' , το οποίο προκύπτει από το G θέτοντας $c'(e) = c(e) - k$, σε χρόνο μικρότερο από τον υπολογισμό της f' από την αρχή.

Για το πρόβλημα αυτό έχουμε δύο περιπτώσεις:

- Αν $f(e) \leq c(e')$ για την προαναφερθείσα ακμή e , τότε η μέγιστη ροή f συνεχίζει να είναι αποδεδειγμένη.
- Αν $f(e) > c(e')$, τότε η ροή f παύει να είναι αποδεδειγμένη, καθώς η ροή $f(e)$ ξεπερνά τη χωρητικότητά της $c'(e)$. Τότε δηλώνεται το πρόβλημα "περίσσεια" ροής $\delta = f(e) - c'(e)$, η οποία πρέπει να μειωθεί, έτσι ώστε η ροή f' να είναι μέγιστη.

και αποδευτή. Η πείραξη αυτή μπορεί να γίνει στο υποδειγματικό πρόβλημα Gf στα μονοπάτια $s-t$ που μπορούν να αποδεχθούν αυτή τη πείραξη. Συγκεκριμένα, ο αλγόριθμος τρέχει ως εξής: Αρχικά ελέγχουμε για το αν η ροή f συνεχίζει να είναι αποδευτή. Αν δεν είναι, υπολογίζουμε το $\delta = f(e) - c'(e)$ που πρέπει να μειωθεί από το δίκτυο ώστε η ροή να είναι αποδευτή. Αφού κατασκευάσουμε το υποδειγματικό δίκτυο Gf για τη ροή f , τρέχουμε ~~unweighted~~ DFS για να βρούμε όλα τα μονοπάτια από το t στο s . Εάν η ροή f είναι μέγιστη, δε θα υπάρχουν $s-t$ μονοπάτια. Στα μονοπάτια αυτά προσπαθούμε να μειώσουμε τη ροή κατά δ , ξεκινώντας από τα μονοπάτια που περιέχουν την αψίδα e . Αν καταφέρουμε να μειώσουμε τη ροή κατά δ προσαρμόζοντας όλα τα μονοπάτια, επιταχύνουμε τη ~~ρ~~ μέγιστη ροή f' . Αν όχι, τότε απαιτείται ο υπολογισμός της f' από την αρχή. Ο αλγόριθμος διατυπώνεται σε ψευδοκώδικα ως εξής:

```

 $\delta \leftarrow f(e) - c'(e);$ 
if  $\delta \leq 0$  then
    return  $(f)$ ;
else
    create  $Gf(V, Ef, rf)$  from  $f$ 
    reduced  $\leftarrow 0$ ;
    run DFS to compute paths from  $t$  to  $s$  in  $Gf$  to  $P$ 
    for each path from  $t$  to  $s$  in  $Gf$  containing  $e$  do
        available  $\leftarrow \min(\min_{e \in p} f(e), \delta - \text{reduced})$ ;
        for all  $e \in p$  do
             $f(e) \leftarrow f(e) - \text{available}$ ; remove
        reduced  $\leftarrow \text{reduced} + \text{available}$ ; remove  $p$  from  $P$ ;
    if reduced  $< \delta$  then
        for all  $p \in P$  do
            available  $\leftarrow \min(\min_{e \in p} f(e), \delta - \text{reduced})$ ;
            for all  $e \in p$  do
                 $f(e) \leftarrow f(e) - \text{available}$ ;
            remove reduced  $\leftarrow \text{reduced} + \text{available}$ ;

```

```

remove p from P;
if reduced =  $\delta$  then
    return( $f'$ );
else
    return (recompilation of  $f'$  needed);

```

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγορίθμου, έχουμε $O(m)$ για την κατασκευή του G_f , $O(m)^{1+\eta}$ για τον DFS και $O(m \cdot \delta)$ για τη μείωση της ροής στα μονοπάτια, καθώς κάθε μονοπάτι μειώνεται τουλάχιστον κατά 1, και αυτό μπορεί να γίνει δ φορές. Επιπλέον, όπως και η επάνοδος, έτσι και η μείωση σε κάθε μονοπάτι απαιτεί $O(m)$. Επομένως, ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(m \cdot \delta)$ όπου $\delta = f(e) - c'(e)$ η οποία είναι σημαντικά μικρότερη από την $O(m^2)$ που απαιτείται για τον υπολογισμό της ροής f' από την αρχή.

Αναφορικά με την ορθότητα του αλγορίθμου, έχουμε τα εξής: Αρχικά, προσαρμογή της ροής απαιτείται μόνο αν $\delta = f(e) - c'(e) > 0$, αλλιώς η ροή είναι αποδεδειγμένη. Ο DFS χρειάζεται να δώσει από τον κόμβο t στον κόμβο s , διότι εφόσον η ροή είναι μέγιστη, δε θα υπήρχαν μονοπάτια $s-t$. Επιπλέον, τα μονοπάτια που έχουν $f(e) > 0$ και δε βρίσκονται από τον DFS δεν μπορούν να μειωθούν. Ξεκινώντας τη μείωση της ροής ~~από~~ κατά το μέγιστο δυνατό ($\min(\delta, \min f_e)$) από τα μονοπάτια που περιέχουν την e , ^{εξ} επαρκεί να πούμε ότι $f'(e) \leq c'(e)$, ώστε η ροή να είναι αποδεδειγμένη. Αυτή η διαδικασία γίνεται μέχρι είτε το δ να μηδενιστεί, το οποίο σημαίνει πως η ροή θα έχει "απαιματοσταθεί" και η νέα ροή f' θα είναι μέγιστη και αποδεδειγμένη, είτε να τελειώσουν τα μονοπάτια και το δ να είναι > 0 , το οποίο σημαίνει ότι απαιτείται υπολογισμός της ροής f' από την αρχή. ~~Από~~ Με βάση τα παραπάνω, συμπεραίνουμε πως ο αλγόριθμος είναι ορθός.

3. Φάχναμε την ελάχιστη ροή $g: l(e) \leq g(e) \leq c(e)$
 $\forall e \in E$ για ένα $s-t$ δίκτυο $G(V, E, l, c)$.

Για το πρόβλημα της μέγιστης ροής f , δημιουργούμε το υποδειγματικό δίκτυο $G_f(V, E_f, r_f)$ όπου οι μπροστά αψές συμβολίζουν τη χωρητικότητα ($r_{uv} = c_{uv} - f_{uv}$) ενώ οι πίσω αψές τη ροή ($r_{vu} = f_{uv}$). Για να βρούμε τη μέγιστη ροή, ημείς θα ταυτοποιήσουμε μια από τις μπροστά αψές σε κάθε $s-t$ μονοπάτι (εμπροσθέντα μονοπάτια).

Το πρόβλημα αυτό είναι την ελάχιστη ροή g . Ως αναλογία με το πρόβλημα της μέγιστης ροής, θα δημιουργήσουμε το υποδειγματικό δίκτυο $G_g(V, E_g, r_g)$ όπου οι μπροστά αψές θα συμβολίζουν τη χωρητικότητα ($r_{uv} = c(e) - l(e)$) και οι πίσω αψές τη ροή ($r_{vu} = g(e) - l(e)$). Στο πρόβλημα αυτό θα προσπαθήσουμε να μεγιστοποιήσουμε τις μπροστά αψές, δηλαδή τη διαθέσιμη χωρητικότητα, και να ελαχιστοποιήσουμε τις πίσω αψές, δηλαδή τη ροή. Αυτό αντιστοιχεί στο να αυξάνουμε τις μπροστά αψές, να μειώνουμε κατά το ίδιο τις πίσω, μέχρι να μην υπάρχει πλέον $t-s$ μονοπάτι. Για να τηρήσουμε τον περιορισμό $g(e) \geq l(e)$, θα προσθέσουμε αρχικά σε όλες τις αψές ροή $g(e) = l(e)$. Υστερα, θα εφαρμόζουμε τον αλγόριθμο blocking flow για να υπολογίσουμε τη μέγιστη $s-t$ ροή του υποδειγματικού δικτύου. Στο τέλος, θα ελέγχουμε για όλες τις αψές την εγκυρότητα. Ο αλγόριθμος αυτός διατυπώνεται σε ψευδοκώδικα ως εξής:

```
for all e  $e \in E$  do  
   $g(e) \leftarrow l(e)$ ;  
construct residual graph  $G_g(V, E_g, r_g)$  with  $r_e^{front} = c(e) - g(e) - l(e)$  and  $r_e^{back} = g(e) - l(e)$ ;  
for all  $u \in V \setminus \{s, t\}$  in  $G_g$  do
```

```

 $excess[u] \leftarrow \sum_{e \in E} g(e) - \sum_{e \in E} g(e);$ 
if  $excess[u] > 0$  then  $c(e) \leftarrow$ 
    add edge  $(s, u)$  with capacity  $excess[u];$ 
else
    add edge  $(u, t)$  with capacity  $-excess[u];$ 
total flow  $\leftarrow 0;$ 
while exists  $t \rightarrow s$  path on  $G_g$  do
    construct level graph  $G'$ ;
    while blocking flow exists in  $G'$  do
        augment blocking flow on  $G'$ ;
        update residual capacities  $G'$ ;
for all  $e \in E$  do
    if  $l(e) \leq g(e) \leq c(e)$  then
        return  $(G_g);$ 
    else
        return (no feasible solution);

```

Στο πρόβλημα αυτό, το υπολεπταίο δίγραφο, στο οποίο οι πρώτοι κόμβοι συμβολίζουν τη χωρητικότητα και οι πύες τη ροή, χρησιμοποιείται για να τρέφουμε αλγόριθμους εύρεσης μέγιστης $s-t$ ροής, δηλαδή να βελτιστοποιήσουμε τη διαθέσιμη χωρητικότητα, το οποίο συνεπάγεται ελαχιστοποίηση της ροής. Αυτό συμβαίνει εφόσον βελτιστοποιείται ο όρος $c(e) - g(e) - l(e)$ για κάθε ακμή, το οποίο συνεπάγεται την ελαχιστοποίηση των όρων $g(e) - l(e)$, το οποίο σημαίνει ότι η ροή g πληραίνει φτάνοντας όσο το δυνατό πιο κοντά στην ελάχιστη επιτρεπόμενη ροή $l(e)$, για όλες τις ακμές. Επιπλέον, καθώς ο αλγόριθμος επαναλαμβάνεται μέχρι να μην υπάρχει $t-s$ μονοπάτι, αυτό σημαίνει πως σε κάθε μονοπάτι θα έχει μινιμίστηκε το άδικο για μια πύα αυτή $g(e) - l(e)$, με αποτέλεσμα να μην μπορεί να μειωθεί περαιτέρω η ροή και να είναι αυτή η απόλυτη. Επομένως, ο αλγόριθμος υπολογίζει πράγματι την ελάχιστη $s-t$ ροή που δέχεται τους περιορισμούς της χωρητικότητας.

Αναφορικά με την υπολογιστική πολυπλοκότητα του αλγόριθμου, έχουμε $O(m)$ για την αρχικοποίηση, $O(m)$ για τον υπολογισμό του $\text{excess}[u] \forall u \in V$, $O(n)$ για την προσθήκη εικονικών αλφών $s' \rightarrow u$ και $u \rightarrow t'$, $O(n)$ για την κατασκευή του ~~BFS~~ level graph (με BFS, $O(m)$) για την εύρεση ενός blocking flow, και το ποσό η επαναλήψεις της διαδικασίας, καθώς μετά από κάθε blocking flow ο αριθμός των επιπέδων μειώνεται. Τέλος, ο έλεγχος της θμυρότητας απαιτεί $O(m)$. Επομένως, ο αλγόριθμος έχει υπολογιστική πολυπλοκότητα $O(n^2 m)$, βέλτιστη για γενικά γραφήλατα.